

Grade 11 ICT: Pascal Programming Complete Guide

Sahan Perera

BComp in Computer Science, USJ (Reading)

How to Use This Book

This document is designed so that **any Grade 11 student can master Pascal programming independently**.

- **Read slowly** - Programming is like learning a new language
- **Type every example** - Don't just read, actually write the code
- **Run the programs** - See what happens when you execute them
- **Experiment** - Change values and see what happens
- **Make mistakes** - Errors teach you more than successes
- **Practice daily** - Even 20 minutes a day is better than 3 hours once a week

Tip

Keep a notebook beside you. Write down new concepts in your own words. This helps your brain remember better!

1 Introduction to Pascal Programming

Pascal is a **structured programming language** created by Niklaus Wirth in 1970. It's named after the French mathematician Blaise Pascal.

1.1 Why Learn Pascal?

- Uses simple, English-like words (**begin, end, if, then**)
- Teaches you to think logically
- Errors are easy to spot and fix
- Foundation for learning other languages like C, C++, Java

1.2 Basic Structure of a Pascal Program

Every Pascal program follows this structure:

```
1 program ProgramName;  
2 { This is where you declare variables }  
3 var  
4     variableName : dataType;  
5  
6 { This is where your actual code goes }  
7 begin  
8     { Your statements here }  
9 end.
```

Example

Your First Pascal Program - Hello World

```
1 program HelloWorld;  
2 begin  
3     writeln('Hello, World!');  
4     writeln('Welcome to Pascal Programming!');  
5 end.
```

Output:

Hello, World!

Welcome to Pascal Programming!

Tip

Notice the period (.) at the very end after **end**. This tells Pascal "the program is complete". Forgetting this period is a common mistake!

2 Identifiers

An **identifier** is simply a *name* you give to things in your program.

Think of it like naming your pet. You could name your dog "Buddy" or "Max". Similarly, you name variables in your program.

2.1 What Can Have Identifiers?

- **Variables** - Storage boxes for values that can change
- **Constants** - Storage boxes for values that never change

- **Programs** - The name of your entire program
- **Procedures and Functions** - Reusable blocks of code (advanced topic)

2.2 Rules for Creating Identifiers

1. **Must start with a letter** (A-Z or a-z)
2. After the first letter, you can use letters, numbers, or underscore (-)
3. **Cannot use spaces**
4. **Cannot use reserved words** (words Pascal already uses)
5. **No special symbols** like @, #, \$, %, &, *, -, +
6. Pascal is **NOT case-sensitive** (Total = total = TOTAL)

2.3 Valid vs Invalid Identifiers

Identifier	Valid?	Reason
studentName	Valid	Starts with letter, no spaces
student_age	Valid	Underscore is allowed
marks2023	Valid	Numbers allowed after first letter
TotalMarks	Valid	Perfectly fine
9thGrade	Invalid	Starts with number
student name	Invalid	Contains space
total-marks	Invalid	Hyphen not allowed
begin	Invalid	Reserved word
student#	Invalid	Special symbol # not allowed
_privateVar	Invalid	Cannot start with underscore

Example

Good Identifier Names:

```

1 var
2   studentAge : integer;
3   averageMarks : real;
4   studentName : string;
5   isPassed : boolean;
6   grade_A_count : integer;

```

Common Mistake

Never use identifiers like `x`, `y`, `a`, `b` unless it's a mathematical program. Use meaningful names like `totalPrice`, `studentCount`. Your code should be readable!

Practice Questions

Identify whether the following are valid or invalid identifiers:

1. `AvgMarks` *(Valid)*
2. `123Total` *(Invalid - starts with number)*
3. `total_sum` *(Valid)*
4. `my-variable` *(Invalid - hyphen not allowed)*
5. `end` *(Invalid - reserved word)*

3 Reserved Words

Reserved words are like **VIP words in Pascal** - they already have special meanings and jobs to do.

3.1 Common Pascal Reserved Words

Word	Word	Word	Word
program	begin	end	var
const	if	then	else
for	to	downto	do
while	repeat	until	case
of	div	mod	and
or	not	true	false
integer	real	string	char
boolean	array	procedure	function

Common Mistake

You cannot use these words as variable names. If you try to write `var begin : integer;`, Pascal will give you an error!

4 Standard Data Types

A **data type** tells Pascal what kind of value a variable can hold. It's like saying "this box can only hold toys" or "this box can only hold books".

4.1 Integer Type

Stores **whole numbers** (positive, negative, or zero).

Example

```
1 var
2   age : integer;
3   temperature : integer;
4   numberOfStudents : integer;
5
6 begin
7   age := 16;
8   temperature := -5;
9   numberOfStudents := 450;
10
11  writeln('Age: ', age);
12  writeln('Temperature: ', temperature);
13  writeln('Students: ', numberOfStudents);
14 end.
```

Output:

Age: 16

Temperature: -5

Students: 450

Tip

Integer range: approximately -32,768 to 32,767 (in standard Pascal)

For larger numbers, some Pascal versions offer `longint`

4.2 Real Type

Stores **decimal numbers** (numbers with fractional parts).

Example

```
1 var
2   height : real;
3   average : real;
4   price : real;
5
6 begin
7   height := 5.8;
8   average := 87.5;
9   price := 1250.75;
10
11  writeln('Height: ', height:0:2, ' feet');
12  writeln('Average: ', average:0:1, '%');
13  writeln('Price: Rs. ', price:0:2);
14 end.
```

Output:

Height: 5.80 feet

Average: 87.5%

Price: Rs. 1250.75

Tip

The notation `:0:2` means "use 0 total width and 2 decimal places". This formats your output nicely!

4.3 Boolean Type

Stores only **True** or **False** values. Think of it as a yes/no switch.

Example

```
1 var
2   isPassed : boolean;
3   isRaining : boolean;
4   hasLicense : boolean;
5
6 begin
7   isPassed := true;
8   isRaining := false;
9   hasLicense := (age >= 18);
10
11   writeln('Passed exam: ', isPassed);
12   writeln('Raining: ', isRaining);
13
14   if isPassed then
15     writeln('Congratulations!')
16   else
17     writeln('Try again next time');
18 end.
```

4.4 Char Type

Stores **exactly ONE character**. Must be enclosed in single quotes.

Example

```
1 var
2   grade : char;
3   initial : char;
4   answer : char;
5
6 begin
7   grade := 'A';
8   initial := 'S';
9   answer := 'Y';
10
11   writeln('Grade: ', grade);
12   writeln('Initial: ', initial);
13
14   if answer = 'Y' then
15     writeln('Yes confirmed');
16 end.
```

Common Mistake

Common mistake: `grade := A;` is WRONG!

Correct: `grade := 'A';` with single quotes

4.5 String Type

Stores **text** (multiple characters). Also enclosed in single quotes.

Example

```
1 var
2   studentName : string;
3   school : string;
4   message : string;
5
6 begin
7   studentName := 'Kamal Perera';
8   school := 'Ananda College';
9   message := 'Welcome to Pascal!';
10
11   writeln('Name: ', studentName);
12   writeln('School: ', school);
13   writeln(message);
14 end.
```

Output:

Name: Kamal Perera
School: Ananda College
Welcome to Pascal!

4.6 Complete Example - All Data Types Together

```
1 program DataTypesDemo;
2 var
3   studentName : string;
4   age : integer;
5   height : real;
6   isPassed : boolean;
7   grade : char;
8
9 begin
10  { Input values }
11  studentName := 'Nimal Silva';
12  age := 16;
13  height := 5.6;
14  isPassed := true;
15  grade := 'A';
16
17  { Display all information }
18  writeln('=== Student Information ===');
```

```

19  writeln('Name: ', studentName);
20  writeln('Age: ', age, ' years');
21  writeln('Height: ', height:0:1, ' feet');
22  writeln('Passed: ', isPassed);
23  writeln('Grade: ', grade);
24  end.

```

5 Variables and Constants

5.1 Variables

Variables are like **storage boxes with labels**. The content can change, but the label stays the same.

Example

Real-Life Analogy:

Think of your water bottle. The bottle (variable) stays the same, but the amount of water (value) keeps changing.

```

1  program VariableExample;
2  var
3      score : integer;
4
5  begin
6      score := 10;
7      writeln('Initial score: ', score);
8
9      score := 20;
10     writeln('Updated score: ', score);
11
12     score := score + 5;
13     writeln('Final score: ', score);
14 end.

```

Output:

Initial score: 10

Updated score: 20

Final score: 25

5.2 Constants

Constants are like **permanent labels** - once set, they NEVER change.

Example

Real-Life Examples of Constants:

- Number of days in a week: 7
- Value of (Pi): 3.14159
- Number of months: 12

```
1 program ConstantExample;  
2 const  
3   PI = 3.14159;  
4   DAYS_IN_WEEK = 7;  
5   PASS_MARK = 50;  
6  
7 var  
8   radius : real;  
9   area : real;  
10  
11 begin  
12   radius := 5.0;  
13   area := PI * radius * radius;  
14  
15   writeln('Radius: ', radius:0:2);  
16   writeln('Area: ', area:0:2);  
17   writeln('Pass mark is: ', PASS_MARK);  
18 end.
```

Tip

Convention: Write constant names in CAPITAL LETTERS. This makes them easy to spot in your code!

6 Input and Output

6.1 Output - Displaying Information

6.1.1 writeln vs write

```

1 program OutputDemo;
2 begin
3     write('Hello ');
4     write('World');
5     { Output: Hello World (on same line) }
6
7     writeln('Pascal');
8     writeln('Programming');
9     { Output: Pascal
10         Programming (on different lines) }
11 end.

```

6.2 Input - Getting Information from User

Example

```

1 program InputDemo;
2 var
3     name : string;
4     age  : integer;
5     height : real;
6
7 begin
8     write('Enter your name: ');
9     readln(name);
10
11     write('Enter your age: ');
12     readln(age);
13
14     write('Enter your height (in feet): ');
15     readln(height);
16
17     writeln;
18     writeln('=== Your Information ===');
19     writeln('Name: ', name);
20     writeln('Age: ', age);
21     writeln('Height: ', height:0:2, ' feet');
22 end.

```

7 Operators

Operators are symbols that tell Pascal to perform specific operations.

7.1 Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	5 + 3	8
-	Subtraction	10 - 4	6
*	Multiplication	6 * 7	42
/	Division (real)	10 / 4	2.5
div	Integer division	10 div 4	2
mod	Remainder	10 mod 4	2

Example

Understanding DIV and MOD:

```
1 program DivMod;
2 var
3     number, divisor : integer;
4     quotient, remainder : integer;
5
6 begin
7     number := 17;
8     divisor := 5;
9
10    quotient := number div divisor;
11    remainder := number mod divisor;
12
13    writeln(number, ' divided by ', divisor);
14    writeln('Quotient: ', quotient);
15    writeln('Remainder: ', remainder);
16    { 17    5 = 3 remainder 2 }
17 end.
```

Output:

17 divided by 5

Quotient: 3

Remainder: 2

7.1.1 Practical Example - Calculator

```

1 program SimpleCalculator;
2 var
3     num1, num2 : real;
4     sum, difference, product, quotient : real;
5
6 begin
7     write('Enter first number: ');
8     readln(num1);
9     write('Enter second number: ');
10    readln(num2);
11
12    sum := num1 + num2;
13    difference := num1 - num2;
14    product := num1 * num2;
15    quotient := num1 / num2;
16
17    writeln;
18    writeln('=== Results ===');
19    writeln('Sum: ', sum:0:2);
20    writeln('Difference: ', difference:0:2);
21    writeln('Product: ', product:0:2);
22    writeln('Quotient: ', quotient:0:2);
23 end.

```

7.2 Comparison (Relational) Operators

These operators compare two values and return True or False.

Operator	Meaning	Example
=	Equal to	5 = 5 is True
≠	Not equal to	5 ≠ 3 is True
>	Greater than	7 > 3 is True
<	Less than	2 < 8 is True
≥	Greater than or equal	5 ≥ 5 is True
≤	Less than or equal	4 ≤ 10 is True

Example

```
1 program ComparisonDemo;
2 var
3     marks : integer;
4
5 begin
6     write('Enter your marks: ');
7     readln(marks);
8
9     writeln('Marks >= 75: ', marks >= 75);
10    writeln('Marks < 35: ', marks < 35);
11    writeln('Marks = 100: ', marks = 100);
12 end.
```

7.3 Logical Operators

Used to combine multiple conditions.

7.3.1 AND Operator

Both conditions must be True for the result to be True.

```
1 program AndExample;
2 var
3     age : integer;
4     hasLicense : boolean;
5
6 begin
7     write('Enter age: ');
8     readln(age);
9     write('Has license (true/false): ');
10    readln(hasLicense);
11
12    if (age >= 18) and (hasLicense = true) then
13        writeln('Can drive!')
14    else
15        writeln('Cannot drive');
16 end.
```

7.3.2 OR Operator

At least one condition must be True for the result to be True.

```
1 program OrExample;
2 var
3     day : string;
4
5 begin
6     write('Enter day: ');
7     readln(day);
8
9     if (day = 'Saturday') or (day = 'Sunday') then
10         writeln('It''s a weekend!')
11     else
12         writeln('It''s a weekday');
13 end.
```

7.3.3 NOT Operator

Reverses the boolean value (True becomes False, False becomes True).

```
1 program NotExample;
2 var
3     isRaining : boolean;
4
5 begin
6     isRaining := false;
7
8     if not isRaining then
9         writeln('Go outside and play!')
10    else
11        writeln('Stay indoors');
12 end.
```

8 Selection Structures (Making Decisions)

Programs need to make decisions based on conditions. This is where selection structures come in.

8.1 IF Statement

The simplest form - do something only if a condition is true.

Example

```
1 program SimpleIf;  
2 var  
3     marks : integer;  
4  
5 begin  
6     write('Enter marks: ');  
7     readln(marks);  
8  
9     if marks >= 50 then  
10        writeln('Congratulations! You passed!');  
11  
12        writeln('Program ended');  
13 end.
```

8.2 IF...ELSE Statement

Do one thing if condition is true, another if it's false.

Example

```
1 program IfElse;  
2 var  
3     marks : integer;  
4  
5 begin  
6     write('Enter marks: ');  
7     readln(marks);  
8  
9     if marks >= 50 then  
10        writeln('You PASSED!')  
11     else  
12        writeln('You FAILED. Try again next time.');
```

8.3 IF...ELSE IF...ELSE (Multiple Conditions)

Example

```
1 program GradeCalculator;
2 var
3     marks : integer;
4
5 begin
6     write('Enter marks (0-100): ');
7     readln(marks);
8
9     if marks >= 75 then
10        writeln('Grade: A - Excellent!')
11    else if marks >= 65 then
12        writeln('Grade: B - Very Good')
13    else if marks >= 55 then
14        writeln('Grade: C - Good')
15    else if marks >= 40 then
16        writeln('Grade: S - Satisfactory')
17    else
18        writeln('Grade: F - Failed');
19 end.
```

8.4 Multiple Statements in IF Block

When you need to execute multiple statements, use `begin...end`.

```
1 program MultipleStatements;
2 var
3     marks : integer;
4
5 begin
6     write('Enter marks: ');
7     readln(marks);
8
9     if marks >= 50 then
10        begin
11            writeln('*** CONGRATULATIONS ***');
12            writeln('You have passed the exam!');
13            writeln('Well done!');
14        end
15    end
```

```

15  else
16  begin
17      writeln('Unfortunately, you did not pass');
18      writeln('Keep studying and try again');
19  end;
20 end.

```

8.5 CASE Statement

When you have many possible values to check, CASE is cleaner than multiple IF statements.

Example

```

1  program MenuSystem;
2  var
3      choice : integer;
4
5  begin
6      writeln('=== Menu ===');
7      writeln('1. New Game');
8      writeln('2. Load Game');
9      writeln('3. Settings');
10     writeln('4. Exit');
11     write('Enter choice: ');
12     readln(choice);
13
14     case choice of
15         1 : writeln('Starting new game...');
16         2 : writeln('Loading saved game...');
17         3 : writeln('Opening settings...');
18         4 : writeln('Goodbye!');
19     else
20         writeln('Invalid choice!');
21     end;
22 end.

```

Example

CASE with Characters:

```
1 program GradeComment;
2 var
3     grade : char;
4
5 begin
6     write('Enter grade (A/B/C/S/F): ');
7     readln(grade);
8
9     case grade of
10         'A', 'a' : writeln('Outstanding performance!');
11         'B', 'b' : writeln('Very good work!');
12         'C', 'c' : writeln('Good job!');
13         'S', 's' : writeln('Satisfactory');
14         'F', 'f' : writeln('Need improvement');
15     else
16         writeln('Invalid grade');
17     end;
18 end.
```

9 Repetition Structures (Loops)

Loops allow you to repeat code multiple times without writing it over and over.

9.1 FOR Loop

Use when you know **exactly how many times** to repeat.

9.1.1 FOR...TO (Ascending)

Example

```
1 program CountingUp;  
2 var  
3   i : integer;  
4  
5 begin  
6   writeln('Counting from 1 to 5:');  
7   for i := 1 to 5 do  
8     writeln(i);  
9 end.
```

Output:

Counting from 1 to 5:

1
2
3
4
5

9.1.2 FOR...DOWNTO (Descending)

Example

```
1 program Countdown;
2 var
3     i : integer;
4
5 begin
6     writeln('Countdown:');
7     for i := 5 downto 1 do
8         writeln(i);
9     writeln('Blast off!');
10 end.
```

Output:

Countdown:

5

4

3

2

1

Blast off!

9.1.3 Practical Example - Multiplication Table

```
1 program MultiplicationTable;
2 var
3     number, i : integer;
4
5 begin
6     write('Enter a number: ');
7     readln(number);
8
9     writeln('Multiplication table of ', number, ':');
10    for i := 1 to 10 do
11        writeln(number, ' x ', i, ' = ', number * i);
12 end.
```

9.1.4 FOR Loop with Multiple Statements

```
1 program SumAndAverage;
```

```

2 var
3   i, number, sum : integer;
4   average : real;
5
6 begin
7   sum := 0;
8
9   for i := 1 to 5 do
10    begin
11      write('Enter number ', i, ': ');
12      readln(number);
13      sum := sum + number;
14    end;
15
16    average := sum / 5;
17
18    writeln('Sum: ', sum);
19    writeln('Average: ', average:0:2);
20 end.

```

9.2 WHILE Loop

Use when you don't know how many times to repeat - continue while a condition is true.

Example

```

1 program WhileExample;
2 var
3   count : integer;
4
5 begin
6   count := 1;
7
8   while count <= 5 do
9     begin
10      writeln('Count: ', count);
11      count := count + 1;
12    end;
13
14    writeln('Loop finished!');
15 end.

```

9.2.1 Practical Example - Password Checker

```
1 program PasswordChecker;
2 var
3     password : string;
4
5 begin
6     password := '';
7
8     while password <> 'secret' do
9         begin
10             write('Enter password: ');
11             readln(password);
12
13             if password <> 'secret' then
14                 writeln('Wrong password! Try again.');
```

9.3 REPEAT...UNTIL Loop

Similar to WHILE, but checks condition at the END (executes at least once).

Example

```
1 program RepeatExample;
2 var
3     number : integer;
4
5 begin
6     repeat
7         write('Enter a positive number: ');
8         readln(number);
9
10        if number <= 0 then
11            writeln('Invalid! Must be positive.');
```

9.4 Difference Between WHILE and REPEAT

```
1 program WhileVsRepeat;
2 var
3     x : integer;
4
5 begin
6     { WHILE - may not execute at all }
7     x := 10;
8     while x < 5 do
9         begin
10             writeln(x);
11             x := x + 1;
12         end;
13     { This prints nothing because x is already 10 }
14
15     { REPEAT - always executes at least once }
16     x := 10;
17     repeat
18         writeln(x);
19         x := x + 1;
20     until x > 5;
21     { This prints 10 once, then stops }
```

22 end.

10 Nested Control Structures

You can put loops inside loops, or IF statements inside loops!

10.1 Nested FOR Loops

Example

Multiplication Table (1-5):

```
1 program MultiplicationChart;
2 var
3     i, j : integer;
4
5 begin
6     for i := 1 to 5 do
7         begin
8             for j := 1 to 5 do
9                 write(i * j:4); { :4 gives spacing }
10                writeln; { New line after each row }
11            end;
12        end.
```

Output:

```
1 2 3 4 5
2 4 6 8 10
3 6 9 12 15
4 8 12 16 20
5 10 15 20 25
```

10.2 Pattern Printing

```
1 program StarPattern;
2 var
3     i, j : integer;
4
5 begin
6     for i := 1 to 5 do
7         begin
```

```

8     for j := 1 to i do
9         write('* ');
10        writeln;
11    end;
12 end.

```

Output:

```

*
* *
* * *
* * * *
* * * * *

```

10.3 IF Inside FOR Loop

Example

Print Even Numbers:

```

1 program EvenNumbers;
2 var
3     i : integer;
4
5 begin
6     writeln('Even numbers from 1 to 20:');
7     for i := 1 to 20 do
8         begin
9             if i mod 2 = 0 then
10                writeln(i);
11            end;
12 end.

```

10.4 Complex Example - Grade Statistics

```

1 program GradeStatistics;
2 var
3     numStudents, i, marks : integer;
4     passCount, failCount : integer;
5     totalMarks : integer;
6     average : real;
7
8 begin

```

```

9   passCount := 0;
10  failCount := 0;
11  totalMarks := 0;
12
13  write('How many students? ');
14  readln(numStudents);
15
16  for i := 1 to numStudents do
17  begin
18      write('Enter marks for student ', i, ': ');
19      readln(marks);
20
21      totalMarks := totalMarks + marks;
22
23      if marks >= 50 then
24      begin
25          passCount := passCount + 1;
26          writeln('Student ', i, ' PASSED');
27      end
28      else
29      begin
30          failCount := failCount + 1;
31          writeln('Student ', i, ' FAILED');
32      end;
33  end;
34
35  average := totalMarks / numStudents;
36
37  writeln;
38  writeln('=== STATISTICS ===');
39  writeln('Total students: ', numStudents);
40  writeln('Passed: ', passCount);
41  writeln('Failed: ', failCount);
42  writeln('Average marks: ', average:0:2);
43 end.

```

11 Complete Programs - Putting It All Together

11.1 Program 1: Simple ATM System

```

1 program SimpleATM;
2 var
3     balance : real;
4     choice : integer;
5     amount : real;
6     pin, enteredPin : string;
7
8 begin
9     balance := 10000.00;
10    pin := '1234';
11
12    { PIN verification }
13    write('Enter PIN: ');
14    readln(enteredPin);
15
16    if enteredPin <> pin then
17    begin
18        writeln('Wrong PIN! Access denied. ');
19        exit;
20    end;
21
22    writeln('PIN correct. Welcome! ');
23    writeln;
24
25    { Main menu }
26    repeat
27        writeln('=== ATM Menu ===');
28        writeln('1. Check Balance ');
29        writeln('2. Withdraw ');
30        writeln('3. Deposit ');
31        writeln('4. Exit ');
32        write('Choice: ');
33        readln(choice);
34
35        case choice of
36            1 : begin
37                writeln('Balance: Rs. ', balance:0:2);
38            end;
39
40            2 : begin
41                write('Amount to withdraw: ');

```

```

42         readln(amount);
43         if amount > balance then
44             writeln('Insufficient funds!')
45         else
46             begin
47                 balance := balance - amount;
48                 writeln('Withdrawn: Rs. ', amount:0:2);
49                 writeln('New balance: Rs. ', balance:0:2);
50             end;
51         end;
52
53     3 : begin
54         write('Amount to deposit: ');
55         readln(amount);
56         balance := balance + amount;
57         writeln('Deposited: Rs. ', amount:0:2);
58         writeln('New balance: Rs. ', balance:0:2);
59     end;
60
61     4 : writeln('Thank you! Goodbye. ');
62
63     else
64         writeln('Invalid choice!');
65 end;
66
67     writeln;
68     until choice = 4;
69 end.

```

11.2 Program 2: Number Guessing Game

```

1 program GuessingGame;
2 var
3     secretNumber, guess, attempts : integer;
4     playAgain : char;
5
6 begin
7     repeat
8         secretNumber := 42; { In real version, use random }
9         attempts := 0;
10

```

```

11  writeln('=== Number Guessing Game ===');
12  writeln('I''m thinking of a number between 1-100');
13  writeln;
14
15  repeat
16      write('Your guess: ');
17      readln(guess);
18      attempts := attempts + 1;
19
20      if guess < secretNumber then
21          writeln('Too low! Try higher.')
22      else if guess > secretNumber then
23          writeln('Too high! Try lower.')
24      else
25          begin
26              writeln('*** CORRECT! ***');
27              writeln('You guessed it in ', attempts, ' attempts!');
28          end;
29
30  until guess = secretNumber;
31
32  writeln;
33  write('Play again? (Y/N): ');
34  readln(playAgain);
35  writeln;
36
37  until (playAgain = 'N') or (playAgain = 'n');
38
39  writeln('Thanks for playing!');
40  end.

```

11.3 Program 3: Student Report Card

```

1  program ReportCard;
2  var
3      name : string;
4      maths, science, english : integer;
5      total : integer;
6      average : real;
7      grade : char;
8

```

```

9 begin
10     writeln('=== Student Report Card System ===');
11     writeln;
12
13     write('Student name: ');
14     readln(name);
15
16     write('Maths marks (0-100): ');
17     readln(maths);
18     write('Science marks (0-100): ');
19     readln(science);
20     write('English marks (0-100): ');
21     readln(english);
22
23     { Calculate total and average }
24     total := maths + science + english;
25     average := total / 3;
26
27     { Determine grade }
28     if average >= 75 then
29         grade := 'A'
30     else if average >= 65 then
31         grade := 'B'
32     else if average >= 55 then
33         grade := 'C'
34     else if average >= 40 then
35         grade := 'S'
36     else
37         grade := 'F';
38
39     { Print report card }
40     writeln;
41     writeln('=====');
42     writeln('          REPORT CARD');
43     writeln('=====');
44     writeln('Student: ', name);
45     writeln('-----');
46     writeln('Subject          Marks');
47     writeln('-----');
48     writeln('Mathematics      ', maths);
49     writeln('Science          ', science);

```

```

50  writeln('English          ', english);
51  writeln('-----');
52  writeln('Total:          ', total);
53  writeln('Average:        ', average:0:2, '%');
54  writeln('Grade:          ', grade);
55  writeln('=====');
56
57  { Pass/Fail decision }
58  if grade = 'F' then
59      writeln('Result: FAIL')
60  else
61      writeln('Result: PASS');
62  end.

```

12 Common Mistakes and How to Avoid Them

Common Mistake

Mistake 1: Forgetting the period at the end

```

1  end  { WRONG - no period }
2  end. { CORRECT }

```

Common Mistake

Mistake 2: Using = instead of :=

```

1  age = 16;      { WRONG }
2  age := 16;     { CORRECT }

```

Common Mistake

Mistake 3: Forgetting semicolons

```

1  writeln('Hello') { WRONG - missing semicolon }
2  writeln('Hello'); { CORRECT }

```

Common Mistake

Mistake 4: Not matching begin and end

```
1 if marks > 50 then
2 begin
3     writeln('Pass');
4     writeln('Good');
5 { Missing end here! }
```

Common Mistake

Mistake 5: Wrong quotes for strings

```
1 name := "John";    { WRONG - use single quotes }
2 name := 'John';    { CORRECT }
```

13 Practice Exercises

13.1 Beginner Level

1. Write a program to calculate the area of a rectangle
2. Write a program to convert Celsius to Fahrenheit
3. Write a program that checks if a number is positive or negative
4. Write a program to find the largest of three numbers
5. Write a program to print your name 10 times

13.2 Intermediate Level

1. Write a program to calculate factorial of a number
2. Write a program to check if a number is prime
3. Write a program to print Fibonacci series up to n terms
4. Write a program to reverse a number
5. Write a program that acts as a simple calculator

13.3 Advanced Level

1. Write a program to create a student database (5 students)
2. Write a program to find sum of digits of a number
3. Write a program to check if a string is a palindrome
4. Write a program for a simple quiz game (5 questions)
5. Write a program to print a pyramid pattern with stars

14 Debugging Tips

Tip

When you get an error:

1. Read the error message carefully
2. Check the line number mentioned
3. Look for:
 - Missing semicolons
 - Mismatched begin-end pairs
 - Wrong variable names
 - Using = instead of :=
4. Use writeln to print variable values and check logic
5. Comment out code sections to isolate the problem

Final Message to Students

You Can Do This!

Programming is NOT about being a genius. It's about:

- **Practice** - Write code every day
- **Patience** - Everyone makes mistakes
- **Persistence** - Don't give up when you get errors
- **Problem-solving** - Break big problems into small steps

Remember:

- Every programmer was once a beginner
- Errors are your teachers, not your enemies
- Google is your friend (search for error messages)
- Practice makes perfect

Start small, dream big!

Quick Reference Guide

Data Type Summary

- `integer` - whole numbers (-32768 to 32767)
- `real` - decimal numbers
- `char` - single character ('A')
- `string` - text ('Hello World')
- `boolean` - true or false

Input/Output

- `readln(variable)` - read input
- `writeln(text)` - print with new line
- `write(text)` - print without new line

Operators

- Arithmetic: + - * / div mod
- Comparison: = < > <= >=
- Logical: and or not

Control Structures

- `if...then...else` - decisions
- `case...of` - multiple choices
- `for...to...do` - counted loops
- `while...do` - condition-based loops
- `repeat...until` - execute at least once

15 Arrays (One-Dimensional Arrays)

In programming, we use **variables** to store data in memory. But what if we want to store **many values of the same data type**?

For example, if you want to store marks of 35 students, writing 35 variable names like this is not practical:

```
1 var m1, m2, m3, ... : integer;    { BAD idea for many values }
```

Tip

When you have **many items of the same type** (like marks, ages, prices), use an **array**. It saves time and keeps your program clean.

15.1 Why Use Arrays? (Use of Arrays)

An **array** stores many values using **one identifier name**, and each value is accessed by an **index**.

Example

Train Analogy:

Imagine a train:

- The train name = array name (example: `marks`)
- Each cart = an element (one data item)
- Cart number = index (0,1,2,...)

So `marks[0]` is the first cart, `marks[1]` is the second, etc.

15.2 Defining a One-Dimensional Array

Correct Pascal syntax:

```
1 var
2   arrayName : array[firstIndex..lastIndex] of dataType;
```

Example

Example: Array for 10 integer marks

```
1 var
2   marks : array[0..9] of integer;
```

This creates **10 elements**: `marks[0]` to `marks[9]`.

Common Mistake

Many students write wrong syntax like `marks ( array[0..9] of integer;` That is incorrect in Pascal.

Correct form is: `marks : array[0..9] of integer;`

15.3 Important Attributes of an Array

- Elements are stored **sequentially** (next to each other in memory).
- Each element has an **index** written inside **square brackets**.
- All values must be of the **same data type**.
- You can access any element directly (random access): `marks[7]` is allowed anytime.
- Arrays work perfectly with loops (`for`, `while`, `repeat`).

Example

Array size depends on index range:

```
1 var
2   A : array[0..4] of integer; { size = 5 elements }
3   B : array[1..5] of integer; { size = 5 elements }
```

Both arrays have 5 elements, but indexes are different.

15.4 Assigning Values to an Array (Manual Assignments)

Example

Assign values to specific elements

```
1 program ArrayAssignDemo;
2 var
3   num : array[0..4] of integer;
4 begin
5   num[0] := 45;
6   num[2] := 36;
7   num[4] := 60;
8   num[1] := num[4] + 15; { 60 + 15 = 75 }
9   num[3] := num[0] + num[2]; { 45 + 36 = 81 }
10
11   writeln('num[0] = ', num[0]);
12   writeln('num[1] = ', num[1]);
13   writeln('num[2] = ', num[2]);
14   writeln('num[3] = ', num[3]);
15   writeln('num[4] = ', num[4]);
16 end.
```

15.5 Displaying Array Values (Printing Elements)

Example

Printing selected elements

```
1 writeln(num[3]); { prints the 4th element }
2 writeln(num[1], ' ', num[4]); { prints 2nd and 5th }
```

Example

Printing using loops (best method)

```
1 program PrintArrayDemo;
2 var
3     num : array[0..4] of integer;
4     x : integer;
5 begin
6     num[0] := 45; num[1] := 75; num[2] := 36; num[3] := 81; num
       [4] := 60;
7
8     writeln('First 4 elements:');
9     for x := 0 to 3 do
10         writeln(num[x]);
11
12     writeln('Elements 3rd to 5th:');
13     for x := 2 to 4 do
14         writeln(num[x]);
15
16     writeln('All elements:');
17     for x := 0 to 4 do
18         writeln(num[x]);
19 end.
```

15.6 Practical Program: ICT Marks of 35 Students (Max + Average)

This is a very common exam-style question: **input marks, find maximum, find average.**

Example

```
1 program ICTMarks(input, output);
2 var
3     marks : array[0..34] of integer;
4     i, tot, max : integer;
5     avg : real;
6 begin
7     tot := 0;
8
9     { Input marks and calculate total }
10    for i := 0 to 34 do
11        begin
12            write('Enter marks of student ', i+1, ': ');
13            readln(marks[i]);
14            tot := tot + marks[i];
15        end;
16
17    avg := tot / 35;
18
19    { Find maximum }
20    max := marks[0];
21    for i := 1 to 34 do
22        if marks[i] > max then
23            max := marks[i];
24
25    writeln;
26    writeln('Maximum marks = ', max);
27    writeln('Average marks = ', avg:0:2);
28 end.
```

Tip

Notice we used `i+1` for display, because array index starts at 0, but student number should start at 1 (student 1, student 2, ...).

16 Sub-Programs (Procedures and Functions)

When a program becomes large, it becomes:

- Hard to read

- Hard to debug
- Hard to maintain

So we break it into smaller parts called **sub-programs**.

16.1 Types of Sub-Programs

- **Procedure:** does a task but **does not return a value**.
- **Function:** does a task and **returns a value**.

Tip

Easy memory trick:

- **Function** gives you a **final answer** (returns value)
- **Procedure** just **does a job** (no return)

16.2 Where to Write Procedures and Functions?

In Pascal, procedures/functions must be declared **before** the main `begin ... end`.

```

1 program MyProgram;
2 const
3     ...
4 var
5     ...
6
7 procedure ...
8 begin
9     ...
10 end;
11
12 function ...
13 begin
14     ...
15 end;
16
17 begin
18     { main program here }
19 end.
```

16.3 Procedure Syntax (No Return Value)

```
1 procedure ProcedureName(parameters);  
2 begin  
3     statements;  
4 end;
```

16.4 Function Syntax (Returns a Value)

```
1 function FunctionName(parameters) : returnType;  
2 begin  
3     statements;  
4     FunctionName := value; { return value }  
5 end;
```

Common Mistake

In Pascal, a function returns a value by assigning to its **own name**:

```
FunctionName := value;
```

Many beginners try `return value`; (that is not standard Pascal).

16.5 Example 1: Circle Program Using Procedures

Example

```
1 program ProcedureCircle(input, output);
2 const
3     PI = 22/7;
4 var
5     radius : real;
6
7 procedure GetData(var r: real);
8 begin
9     write('Enter radius: ');
10    readln(r);
11 end;
12
13 procedure ProcessArea(r: real);
14 var
15     area : real;
16 begin
17     area := PI * r * r;
18     writeln('Area = ', area:0:2);
19 end;
20
21 procedure ProcessCircumference(r: real);
22 var
23     circum : real;
24 begin
25     circum := 2 * PI * r;
26     writeln('Circumference = ', circum:0:2);
27 end;
28
29 begin
30     GetData(radius);
31     ProcessCircumference(radius);
32     ProcessArea(radius);
33 end.
```

16.6 Example 2: Circle Program Using Functions

Example

```
1 program FunctionCircle(input, output);
2 const
3     PI = 22/7;
4 var
5     radius : real;
6
7 function Area(r: real) : real;
8 begin
9     Area := PI * r * r;
10 end;
11
12 function Circumference(r: real) : real;
13 begin
14     Circumference := 2 * PI * r;
15 end;
16
17 begin
18     write('Enter radius: ');
19     readln(radius);
20
21     writeln('Circumference = ', Circumference(radius):0:2);
22     writeln('Area = ', Area(radius):0:2);
23 end.
```

Tip

When to use what?

- Use **function** when you need a value back (like area, max, sum).
- Use **procedure** when you just want to perform steps (like printing a menu, reading input).

17 Evolution of Programming Languages

A computer needs instructions to do tasks. A **program** is a sequence of instructions written to solve a problem.

But computers do not naturally understand English. They understand only machine-

level instructions. That is why programming languages evolved.

17.1 Low-Level Languages

17.1.1 Machine Language

Machine language is written in **binary** (0 and 1). The CPU can execute it directly.

- **Fast**
- **No translator needed**
- **Very hard for humans**
- **Machine dependent** (one computer's machine code may not run on another)

17.1.2 Assembly Language

Assembly uses **symbols** / **mnemonics** instead of 0 and 1 (like MOV, ADD).

- Easier than machine language
- Needs an **assembler** to translate into machine code
- Still machine dependent

Example

Translation idea:

Assembly program → Assembler → Machine code

17.2 High-Level Languages

High-level languages use more human-friendly words and structure.

Examples: **FORTRAN**, **BASIC**, **COBOL**, **PASCAL**, **C**

- Easier to read and write
- Usually machine independent
- Must be translated to machine code before running

Tip

Pascal is a high-level language and is excellent for learning **structured** / **procedural programming**.

18 Programming Paradigms (Types of Programming)

Different languages follow different **approaches** to solving problems. These approaches are called **paradigms**.

18.1 Procedural vs Declarative

18.1.1 Procedural Paradigm (How to do it)

Procedural programming explains **step-by-step how** to solve a problem. It uses statements like **if**, **for**, **while**, procedures, etc.

Example

Procedural idea:

“First do this, then do that, then finish.”

Pascal is mainly procedural.

18.1.2 Declarative Paradigm (What you want)

Declarative programming describes **what result you want**, not the step-by-step method. The system decides how to achieve it.

Example

Declarative idea:

“I want students with marks > 75” (and the system finds them).

Common in AI and database thinking.

Common Mistake

Do not confuse these:

- Procedural: **how to achieve it**
- Declarative: **what you want**

18.2 Structured vs Object-Oriented

18.2.1 Structured Programming

Structured programming focuses on breaking a system into **functions/processes** and uses a **top-down** approach.

Typical flow:

- Input

- Process
- Output

Pascal supports structured programming strongly.

18.2.2 Object-Oriented Programming (OOP)

OOP is based on **objects**. An object has:

- **Data** (attributes / properties)
- **Methods** (actions / procedures that work on data)

Example

Activity (Exam-friendly): Car as a Class

Class: Car

Properties: color, brand, speed, fuelLevel

Methods: start(), stop(), accelerate(), brake()

Objects: ToyotaCar, BMWCar, HondaCar

18.3 Programming vs Scripting

Programming languages often require **compilation** and have strict structure. **Scripting languages** are often **interpreted** and frequently used for automating tasks.

Example

Examples:

- Programming: Pascal, C, C++
- Scripting: JavaScript, PHP

19 Language Translators (Assembler, Interpreter, Compiler)

Most languages (except machine language) need translation into machine code.

19.1 Assembler

Converts **assembly language** into **machine code**.

Example

Assembly → Assembler → Machine code

19.2 Interpreter

An interpreter translates and executes the program **line by line**.

- Executes step-by-step
- If an error occurs, it stops at that line (can run until the error line)
- Translation happens **every time you run** the program

Tip

Interpreters are great for learning because you can quickly test small parts.

19.3 Compiler

A compiler translates the **entire program at once** into machine code (before running).

- If there are errors, the compilation fails and the program does not run
- If there are no errors, you get machine code/executable output
- After compiling once, you can run many times without translating again (unless source code changes)

Example

High-level program → Compiler → Machine code
Then you run the machine code many times.

Common Mistake

Key difference (must know for exams):

- Interpreter: translates **statement by statement**
- Compiler: translates **whole program at once**

20 Practice Questions (Arrays + Subprograms + Translators)

Arrays

1. Declare an integer array to store ages of 20 students (use indexes 1..20).
2. Write a program to input 10 numbers into an array and print them in reverse order.
3. Write a program to input marks of 30 students and find:
 - the highest mark
 - the lowest mark
 - the average

Procedures and Functions

1. Write a **procedure** that prints “Welcome to ICT”.
2. Write a **function** that returns the square of an integer.
3. Write a program using a function to return the maximum of two numbers.

Language Translators

1. Define: assembler, compiler, interpreter.
2. Write 2 differences between interpreter and compiler.
3. Give one advantage and one disadvantage of machine language.